REPORT

TASK 15 - DARN SQL INJECTION

Submitted by:

Neethu Mohan

TABLE OF CONTENTS

1. **Introduction…… 1**
2. **Installation & Environment Setup…… 1**
3. **Low Security Level…….. 3**
4. **Medium Security Level…… 3**
5. **High Security Level…….. 4**
6. **Conclusion…….. 5**

1. INTRODUCTION

This report documents the successful exploitation of SQL Injection vulnerabilities within the Damn Vulnerable Web Application (DVWA). SQL Injection is a critical vulnerability where an attacker interferes with the queries an application makes to its database, allowing the unauthorized retrieval of sensitive data.

2. INSTALLATION & ENVIRONMENT SETUP

The lab was deployed on a **Kali Linux** host using **Docker** to ensure environment isolation and consistency.

1. **Update Repository:** `sudo apt update`
2. **Install Docker Engine:** `sudo apt install docker.io -y`
3. **Start Service:** `sudo systemctl enable --now docker`
4. **Deploy DVWA Container:** `sudo docker run --rm -it -p 80:80 vulnerables/web-dvwa`

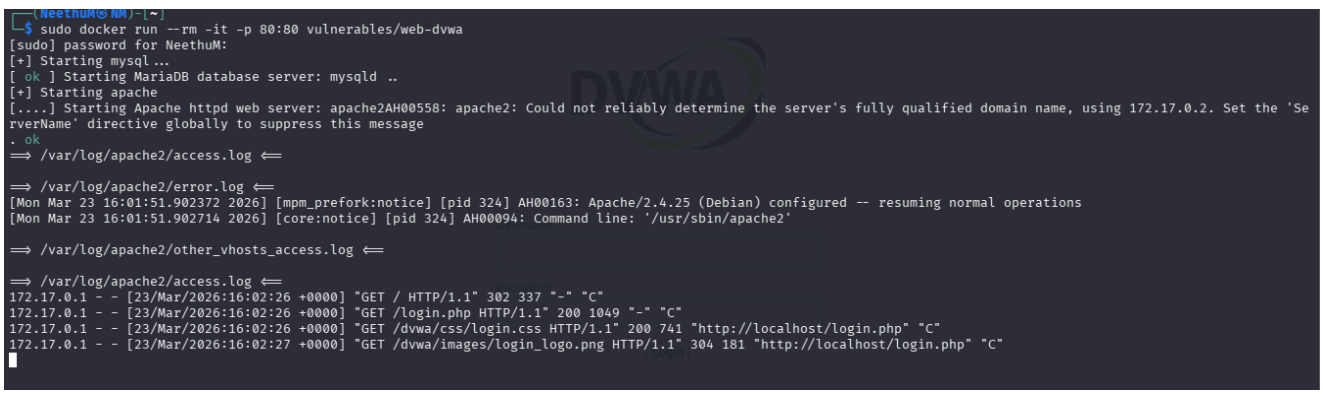

Fig: DVWA Container

5. **Initialization:** * Accessed `http://localhost` via Firefox.

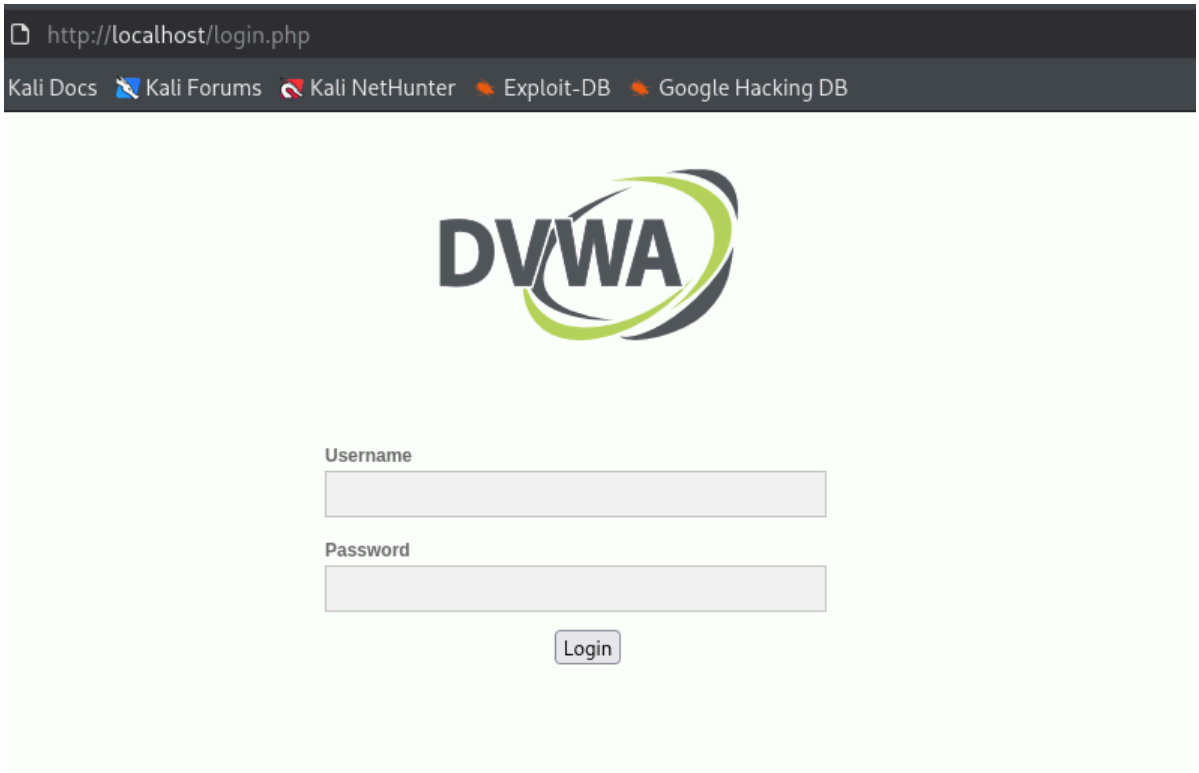

Fig: Login Page

- Authenticated with `admin` / `password`.
- Executed **"Create / Reset Database"** to populate SQL tables.

Fig: Home page

3. LOW SECURITY LEVEL

Objective: Bypass the login-less ID field to extract the full user database.

Technical Analysis: The backend does not sanitize the `$id` variable. By inputting a single quote `'`, we terminate the string and append a `UNION SELECT` statement.

Payload: `1'OR '1'='1'#`

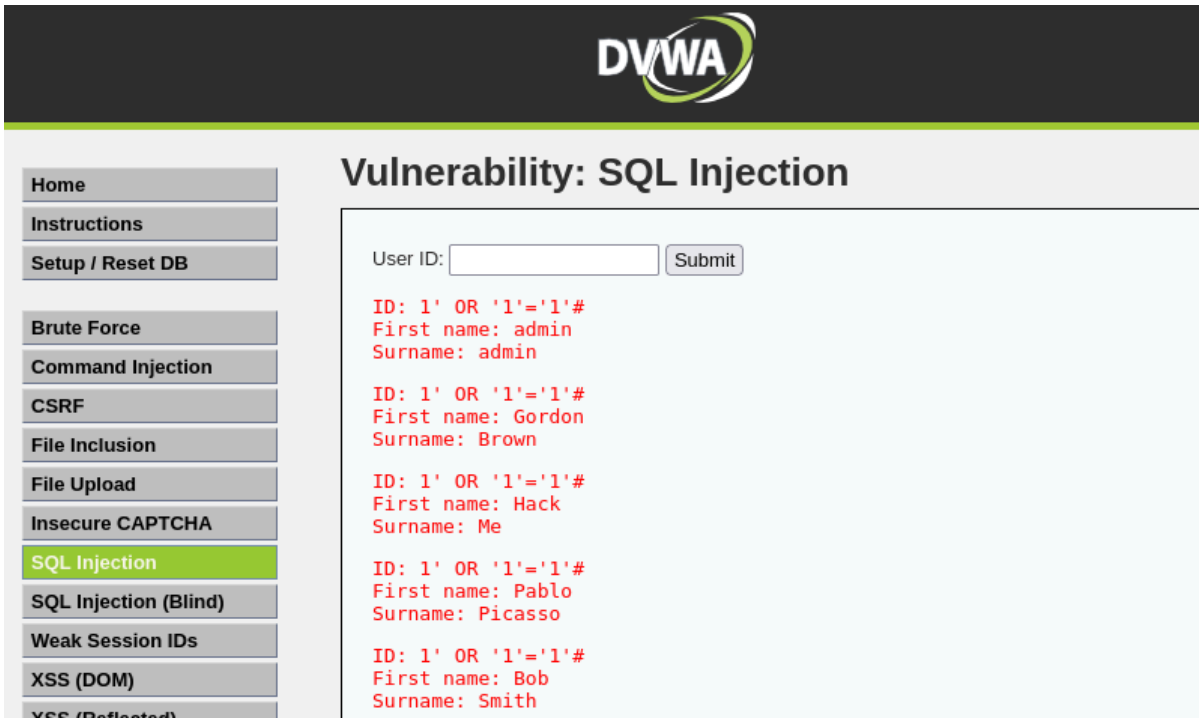

Fig: Low Security

4. MEDIUM SECURITY LEVEL

Objective: Bypass a dropdown menu restriction and single-quote escaping.

Technical Analysis: The developer implemented a dropdown to prevent typing. By using the **Inspect Element** tool, the HTML `value` was manually changed. Since the backend treats the input as an integer, no quotes were required for the injection.

Payload: `1 UNION SELECT user, password FROM users #`

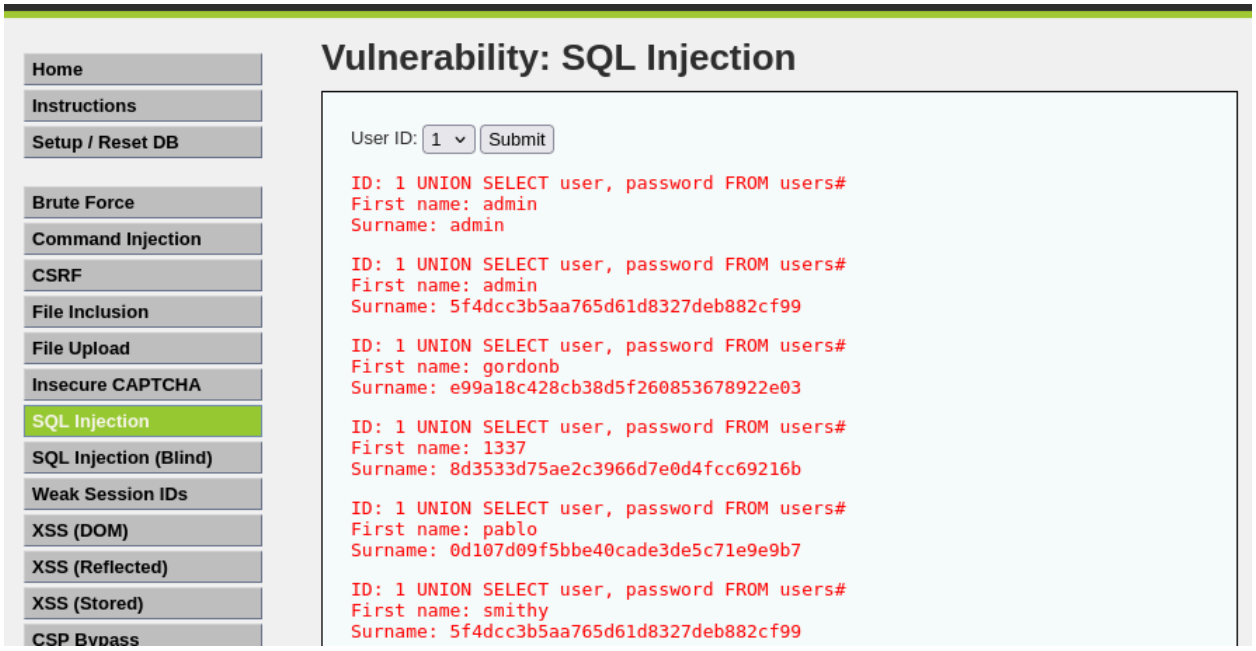

Fig: Medium Security

5. HIGH SECURITY LEVEL

Objective: The goal is to successfully bypass the server-side `LIMIT 1` restriction using SQL comment characters (`#`) to exfiltrate multiple data rows from the backend database.

Technical Analysis: The input is sent via a popup after clicking “Change ID”. The payload includes a `#` (comment) to ignore any `LIMIT` or additional SQL logic appended by the developer.

Payload: `1' AND SUBSTRING(database(),2,1)='v' #`

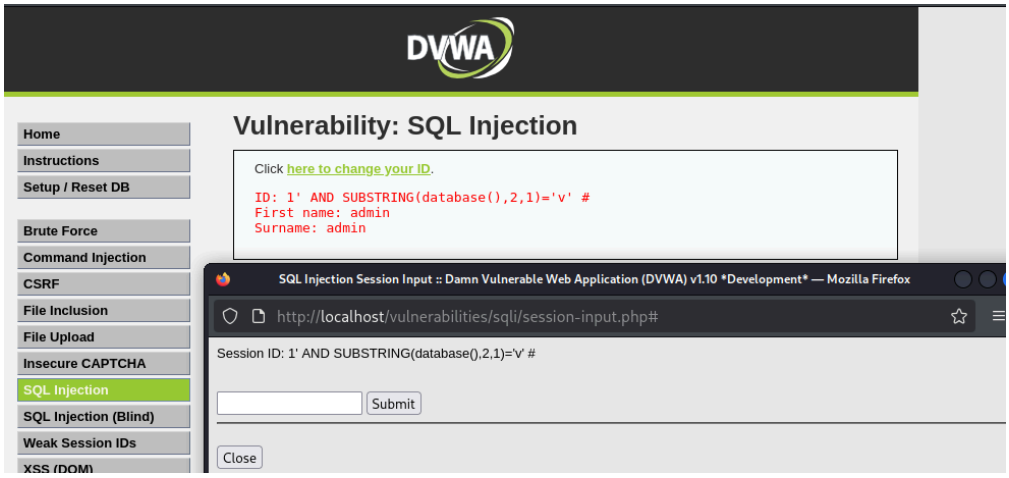

Fig: High Security

CONCLUSION:

The successful exploitation of all three levels confirms that SQL Injection remains a viable threat regardless of the user interface's complexity. The transition from simple UNION-based attacks in Low security to more advanced Boolean-based Blind SQLi in High security proves that as long as user input is concatenated directly into SQL strings, the data remains at risk. The only robust remediation identified is the total abandonment of dynamic SQL in favor of Prepared Statements. The findings of this lab emphasize that developers must treat all user-supplied data as untrusted and utilize secure coding frameworks to prevent unauthorized database interaction.